Predicting Movie Box Office During The Development Stage

Yohanan Ben-Gad and Kabhilesh Giri

Abstract

The most enduring challenge for Hollywood has always been to predict which movies will be successful and which ones won't. In our project, we set out to build a model to predict how much money a movie will make at the box office based on a variety of factors including runtime, budget, genres, cast, and plot keywords. Crucially, unlike previous models we did not include factors such as reviews or user ratings, as these are unknown variables at the time of a movie's development and would therefore make the model less useful to producers when deciding on whether to greenlight a project.

Datasets

We used two Kaggle Datasets: Movies Daily Update Dataset and TMDB-Data-Analysis. We combined columns from both to create a new dataset with the following columns:

```
[movie_id, title, genres, overview,
popularity, production_companies,
production_countries, spoken_languages,
release_date, budget, revenue, runtime,
status, tagline, vote_average, vote_count,
credits, keywords, poster_path, backdrop_path,
recommendations, original_language]
```

Dataset Enrichment

The following columns were introduced via IMDB APIs:

```
[Actor1_Popularity, Actor2_Popularity,
Actor3_Popularity, mpaa_rating]
```

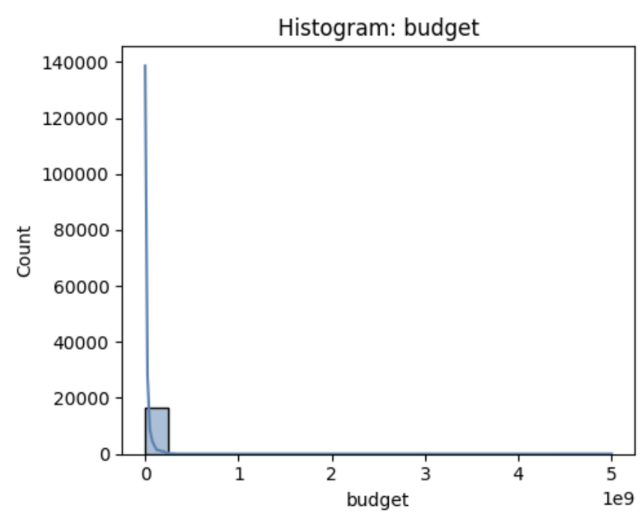
We believed that actor popularity would be hugely important, as conventional wisdom in the movie industry has dictated that people are more likely to see a film if it stars a person they like. MPAA rating is also potentially a big indicator of box-office, with studios being historically reluctant to greenlight expensive R-rated movies due to the limitations on their potential audience.

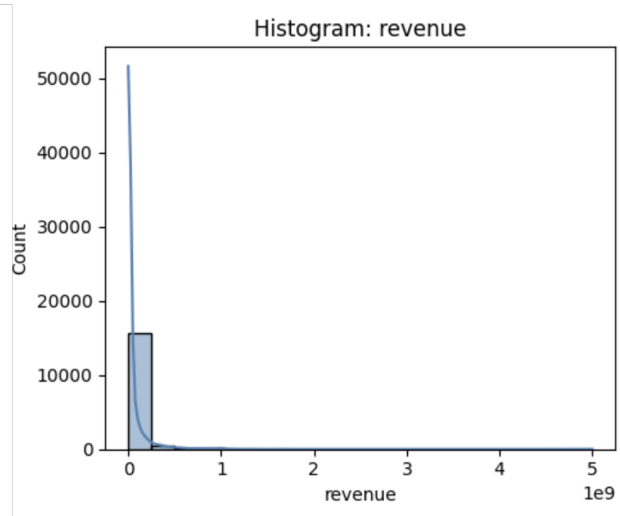
The dataset originally included 652827 samples (representing movies that were featured in both source datasets). However, upon some rudimentary analysis- we discovered that the majority of the samples didn't contain an entry for box-office revenue, making them unusable to us. In addition, we chose to eliminate some of the oldest movies from the dataset (from before 1950) as we felt they would not be indicative of modern trends. After doing these reductions, our number of rows was 16441- an unfortunately significant decrease.

Exploratory Data Analysis

Numerical

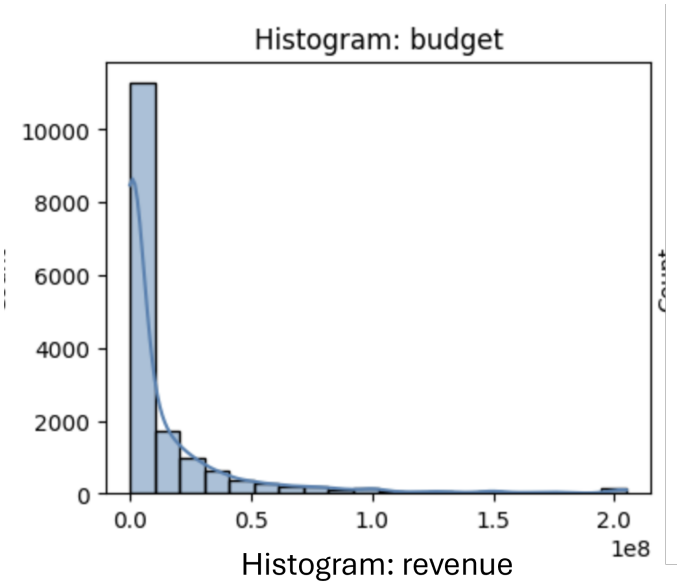
When it came to budget and revenue, we were dealing with an extremely heavy right skew in the data.

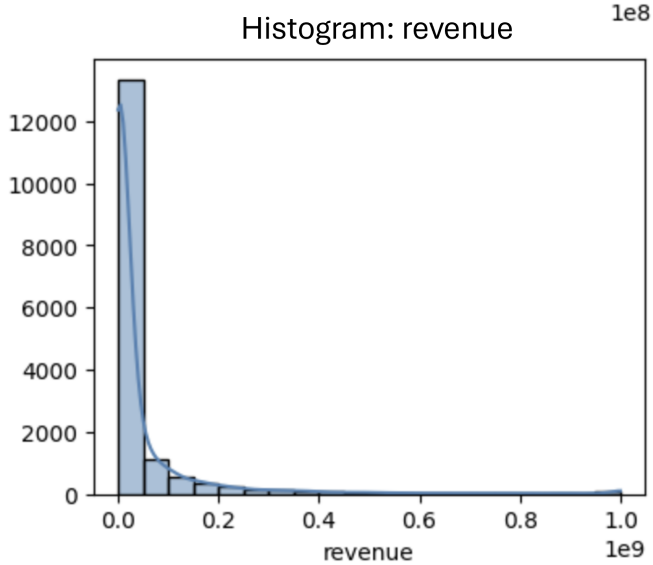




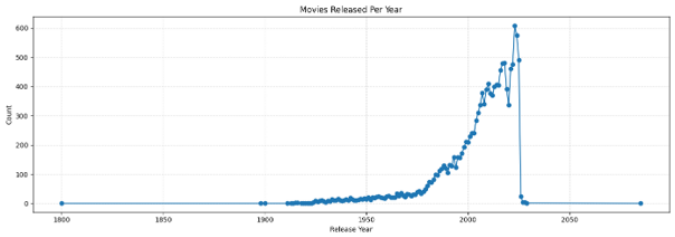
After capping the 99.5 Percentile of the data to the maximum, we were able to make the data distribution a little more manageable.

	feature	cap_quantile	cap_value	applied_on
0	popularity	0.995	2.943929e+02	X_capped
1	budget	0.995	2.050000e+08	X_capped
2	runtime	0.995	2.838050e+02	X_capped
3	revenue	0.995	1.000000e+09	y_capped



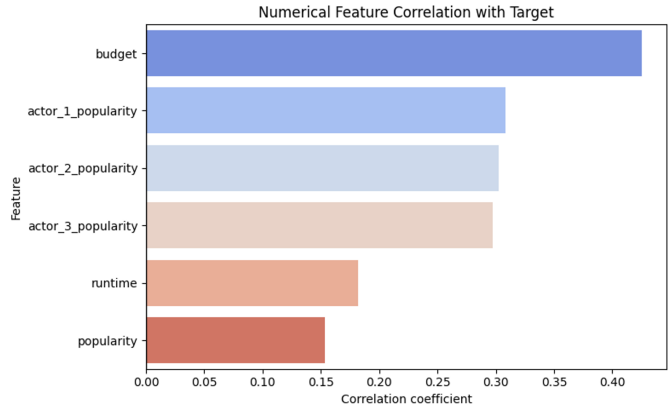


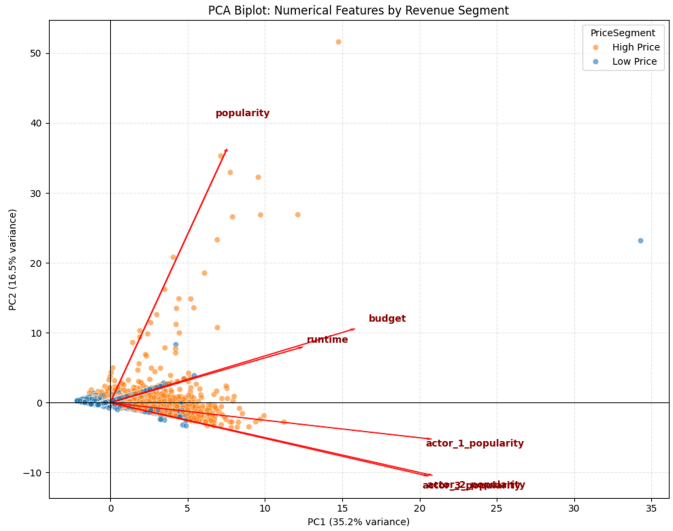
Looking into the release years, we can see that the majority of films in our dataset are more recent, with movies made since 2000 outnumbering those made before (even when accounting for the removed data for movies before 1950) . This is helpful for us, as more recent movies are likely to be more indicative of future predictions than older ones.



The data still included a one notable garbage value- a movie supposedly released in 2050. Given that this investigation was being done in 2026, we opted to remove it.

Our strongest signal predictors were the budget, runtime, and the actor's popularity. We opted to remove the "popularity" (not actor popularity) for two main reasons: this would be an unknown variable during the development stage of a movie and because the popularity value could be as much consequence of a movie's box office as it is a contributor to it. We therefore felt it would be cheating to include it.





Much of our data came with missing values. We first attempted the following imputation strategy to address this:

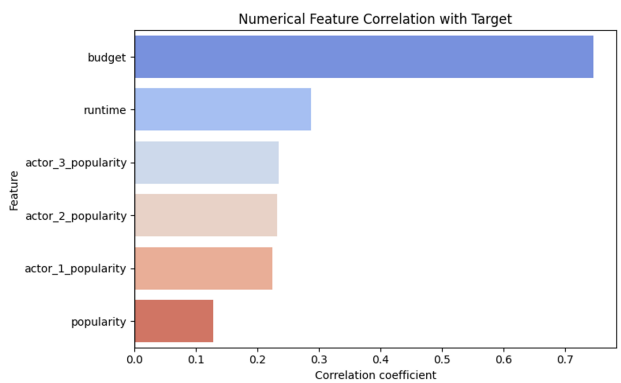
- **Actor Popularity:** Fill missing values with 0 (these are likely cases where actors are too unknown to have an assigned popularity score)
- **Original Language:** Fill missing values with the mode (English) and apply one-hot encoding.
- **MPAA Rating:** Fill missing values with mode ("R") and apply one-hot encoding
- **Textual Features** (keywords, tagline, production_countries, production_companies, genres, overview): Converted to an empty string.

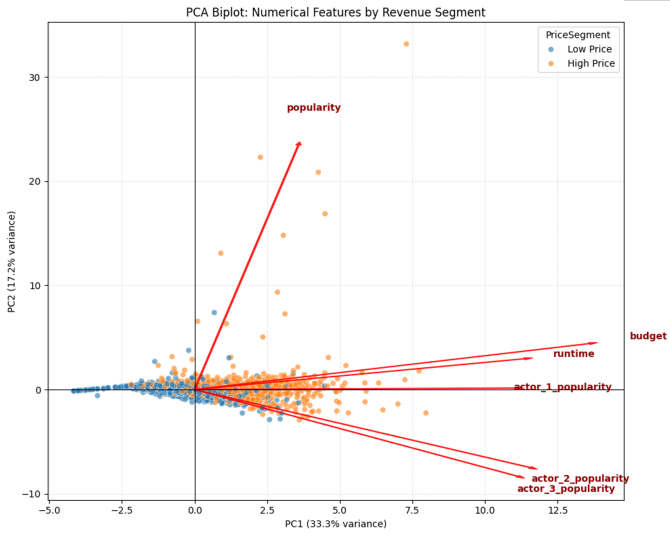
In addition, we created another dataset where we simply removed all the rows with missing values. Running a few preliminary tests gave us these results:

	Data with Imputation (Train/Test)	Data with problematic rows removed (Train/Test)
Linear Regression	0.87/0.56	0.89/0.51
Random Forrest	0.91/0.55	0.90/0.57
XGBoost	0.95/0.54	0.95/0.54

As we can see, there is very little difference between the two sets of results, and no clear pattern as to which of the two was better. In the interest of simplicity, we thus chose to continue working with the smaller dataset (now only 3856 rows).

Unsurprisingly, most of our dataset now came from Hollywood (it was to be expected that there would be more missing data for foreign productions). Unfortunately, the problem of right-skewness persisted, so we once again capped all the values above the 99.5th percentile. Additionally, we applied the log transformation to revenue and budget to ensure all values fit nicely within our scale. We were able to end up with a much cleaner dataset than before:





Categorical and Text

For the text values (overview, keywords, title, tagline), we ran an experiment to see if our results would be improved by clustering.

Originally- we tried grouping keywords and overview values together, as these both serve a similar function in terms of offering insight into the content of a movie. To do this, we built a custom graph-based community detection clustering algorithm.

```
def get_co_occurrence(input_words):
    co_occurrence = defaultdict(int)
    for word in input_words:
        if not word:
            continue
        for k1, k2 in
             itertools.combinations(set(word), 2):
            pair = tuple(sorted([k1, k2]))
            co_occurrence[pair] += 1
    return co_occurrence

def get_clusters(input_words, weight_val):
    co_occurrence = get_co_occurrence(input_words)
    G = nx.Graph()
    for (k1, k2), weight in co_occurrence.items():
        if weight >= weight_val: # threshold
             (important!)
            G.add_edge(k1, k2, weight=weight)
    communities = greedy_modularity_communities(G)
    return [list(c) for c in communities]
```

Unfortunately, after running the combined datasets through the algorithm, we were not able to find any word clusters that showed any visible patterns.

We therefore repeated this process with only the keyword values. The results were initially very promising, with the algorithm outputting clusters that each pointed to a particular type of movie. Here are samples from our first three clusters (each containing hundreds of word entries):

- **Cluster-1:**
  Action/Adventure/Fantasy/Science-Fiction/Blockbuster
  `['pirate ship', 'exotic island', 'extraterrestrial', 'princess', 'queen elizabeth i', 'cat', 'mermaid', 'warlock', 'american civil war', 'parkour',...]`
- **Cluster-2:**
  Crime/Dark Drama/Noir
  `['crime family', 'ambush', 'liberation of hostage', 'cancer', 'hippie', 'true crime', 'mafia boss', 'fbi', 'theft', 'newspaper',...]`
- **Cluster-3:**
  Coming of age/Family Drama/Teen Drama/Class Drama
  `['big family', 'chauffeur', 'final exam', 'based on memoir or autobiography', 'sibling relationship', 'family conflict', 'racehorse', 'comeback', 'nervous breakdown', 'half',...]`

Unfortunately, the size of the clusters reduced very rapidly. By the ninth cluster- there were only 9 word entries total:

- **Cluster-9:**
  `["wrestling', 'pro wrestling', 'female wrestler', 'wrestler', 'qoc', 'professional sports', 'wrestling match', 'female revolution', 'women fight']`

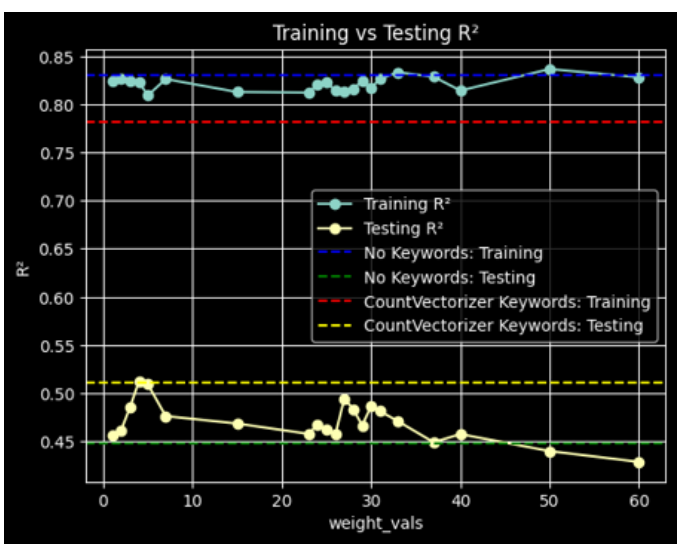
This only covers movies about wrestling, and is likely far too specific to be useful for classification.

Using the K-Means clustering algorithm made this problem even worse. Here are some of the cluster sizes from testing multiple K-values in the range of 5 to 100:

- **K=5:**
  `[17227, 14, 6, 1, 1]`
- **K=10:**
  `[17213, 10, 7, 13, 1, 1, 1, 1, 1, 1]`
- **K=20:**
  `[17170, 1, 50, 1, 6, 5, 2, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1]`

As we can see, nearly every word value is always grouped together in the first cluster, with only a small handful of remainders be distributed among the others.

While our original clustering algorithm was not perfect, we nevertheless decided to test it out. We created two simple control datasets- one without any text values, and one with the keyword values processed through the standard CountVectorizer procedure. We then ran our clustering algorithm with multiple weight thresholds and for each one created a set of columns in our datasets whose value corresponds to the number of unique words in each entry's keywords set present in the columns respective cluster. We then compared the R^2 results:



Unfortunately, we can see that we were never able to exceed the R^2 values attained simply by using CountVectorizer. As such, we determined that using word clustering wasn't going to work and opted to simply use TF-IDF and CountVectorizer for our text columns instead.

Final Breakdown

Field	Type	Processing
title	Text	TF-IDF
release_date	Date	Std. Scaling (Year)
budget	Num.	Std. Scaling
runtime	Num.	Std. Scaling
actor_1_popularity	Num.	Std. Scaling
actor_2_popularity	Num.	Std. Scaling
actor_3_popularity	Num.	Std. Scaling
production_countries	Cat. (multi)	MultiLabelBinarizer
production_companies	Cat. (multi)	MultiLabelBinarizer
genres	Cat. (multi)	MultiLabelBinarizer
spoken_languages	Cat. (multi)	MultiLabelBinarizer
original_language	Cat. (single)	One-Hot
mpaa_rating	Cat. (single)	One-Hot
keywords	Cat. (multi)	CountVec
tagline	Text	TF-IDF
overview	Text	TF-IDF

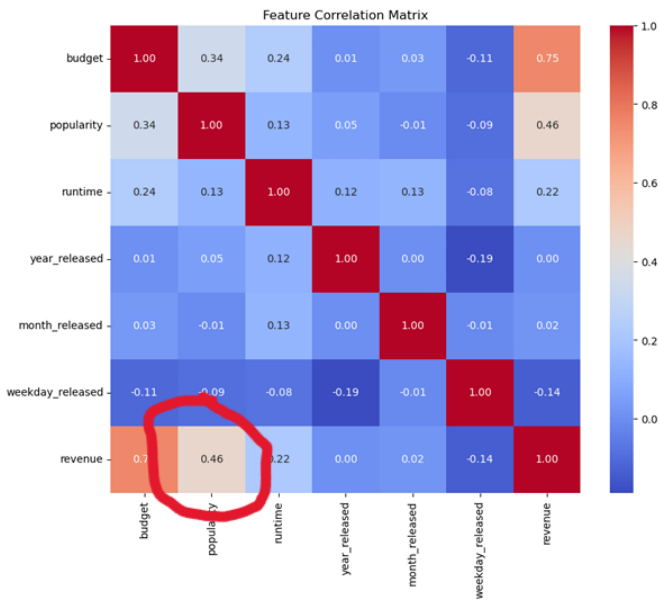
Model Selection and Hyperparameter Tuning

For this task, we conducted 5-Fold Cross Validation on 11 models. We used the Northeastern University CPU Cluster with 26 Cores and 42 GB RAM, which were extremely helpful in conducting an extensive grid search using hyper parameter tuning. The process took between 14 and 16 hours.

Results and Evalutation

Model	Cand.	CV Fits	Train R^2	Test R^2
Linear Regression	720	3600	0.917	0.583
CatBoost	20	100	0.618	0.563
XGBoost (No Lasso)	96	480	0.985	0.595
KNN (No Lasso)	24	120	1.000	0.594
Random Forest (No Lasso)	54	270	0.925	0.585
Elastic Net (No Lasso)	18	90	0.616	0.574
Ridge (No Lasso)	6	30	0.616	0.574
Elastic Net (With Lasso)	198	990	0.605	0.563
Ridge (With Lasso)	66	330	0.607	0.561
KNN (With Lasso)	264	1320	0.999	0.544
Random Forest (With Lasso)	594	2970	0.691	0.236

When analyzing the results, we can see that the XGBoost and KNN models without Lasso are the best performing on the test data. However- we were always at least slightly below an R^2 value of 0.6. This is notably lower than the results obtained by Zheng et al. (2024)[1], who were able to get an R^2 value of above 0.7 when using XGBoost, Bidirectional LSTM and Simple RNN. One thing to note however is that they included the film popularity metric in their input dataset, which we specifically chose to remove as we felt this would make the problem too easy. Supporting the theory that this parameter was responsible for the difference was this feature correlation matrix made by Zheng et al.- showing a high level of correlation between popularity and revenue:



Here are the highest value weights produced by our linear regression model:

Feature	Weight
Marvel Studios	2.3118
title_wars	2.1047
lang_jv	-1.9682
Lucasfilm Ltd.	1.6004
Heyday Films	1.5397
title_spider	1.0116
Eon Productions	0.7949
title_harry	0.7679
Pixar	0.7650
Temple Hill Entertainment	0.7563
20th Century Fox Animation	0.7179
tagline_living	-0.6183
Budget	0.6182

As we can see most of the top weights indicate if a movie is part of a major popular franchise (Marvel, Harry-Potter, Star Wars, James Bond...). This is unfortunately not particularly useful, as it is both extremely obvious that these movies will be successful, and is not scalable to the vast majority of movies that are not attached to a major intellectual property.

Future Work

With the actor popularity and to a lesser extend the mpaa rating metrics proving to be useful, further explorations is warranted into any more features that could provide insightful information which are not available on most online datasets.

Text processing is an area that we have also not explored sufficiently. Other forms of grouping exist such as TruncatedSVD, which could prove to be useful in extracting relevant information.

More attention also needs to be payed to how budget can effect predictions. With a high budget film being likely to gross more than a low budget one, the window for what constitutes a 'realistic' prediction widens. For example the upper and lower limits for what a movie that cost 300 million dollars would be expected to make could be billions of dollars. The equivalent window for a movie that cost lest than 1 million dollars would be 10s of millions at most. This factor obviously has a huge impact on the reliability and usefulness of the model, and further work should be done to address this. One good place to start would be to divide the data into budget ranges and train and test these groups individually.

Finally- we made some initial attempts at using a neural network, with terrible results. Further investigation needs to be conducted into where this model went wrong and how it could potentially be leverage in the future.

References

[1] https://www.researchgate.net/publication/385334208_Predicting_Movie_Box_Office_Based_on_Machine_Learning_Deep_Learning_and_Statistical_Methods